

Proyecto de ejemplo: Gestión de Ventas y Pruebas de Calidad

Curso de posgrado: **Estándares y Métricas de Calidad** – Semana 8

1. Contexto didáctico

En la Semana 8 del curso de especialización en Desarrollo de Software, asignatura **“Estándares y Métricas de Calidad”**, el objetivo es que los estudiantes implementen aseguramiento de calidad mediante:

- Pruebas **unitarias**
- Pruebas **de integración**
- Pruebas **de aceptación** (enfoque BDD: Given-When-Then)

Este proyecto de ejemplo en Java modela un pequeño servicio de **gestión de ventas** con operaciones CRUD sobre una tabla `ventas`. Sobre esta base se construyen las tres suites de pruebas, cada una con un propósito y alcance diferente.

2. Definición de la base de datos: `bd_ventas.sql`

La base de datos `bd_ventas.sql` define una tabla `ventas` que almacena información básica de una venta:

- `id_venta`: identificador único de la venta.
- `fecha`: fecha de la venta.
- `cliente`: nombre del cliente.
- `producto`: nombre del producto.
- `cantidad`: cantidad vendida.
- `precio_unitario`: precio por unidad.
- `total`: total calculado de la venta (`cantidad * precio_unitario`).

Esta tabla sirve como base para demostrar operaciones CRUD y pruebas sobre lógica de negocio.

```
```sql
-- bd_ventas.sql
-- Script de creación de la base de datos de ejemplo para gestión de ventas.
```

```
CREATE TABLE IF NOT EXISTS ventas (
 id_venta INT PRIMARY KEY AUTO_INCREMENT,
 fecha DATE NOT NULL,
 cliente VARCHAR(100) NOT NULL,
 producto VARCHAR(100) NOT NULL,
 cantidad INT NOT NULL,
 precio_unitario DECIMAL(10,2) NOT NULL,
 total DECIMAL(10,2) NOT NULL
);
```

```
-- Comentario didáctico:
-- La columna 'total' se almacena explícitamente para ilustrar la lógica de negocio
-- que calcula total = cantidad * precio_unitario en la capa de servicio.
-- En un diseño más avanzado, podría calcularse dinámicamente, pero aquí
-- se usa para mostrar pruebas sobre el cálculo.
```

### 3. Arquitectura de la aplicación Java de referencia

La aplicación se organiza en capas para ilustrar buenas prácticas:

- **Capa de dominio:** clase `Venta`.
- **Capa de repositorio:** interfaz `VentaRepository` y clase `JdbcVentaRepository`.
- **Capa de servicio:** clase `VentaService` con lógica de negocio (cálculo de total, validaciones básicas).
- **Capa de presentación mínima:** clase `Main` que muestra un flujo simple de uso.

Se asume un proyecto Maven con dependencias básicas:

```
<!-- Fragmento de pom.xml (referencial) -->
<dependencies>
 <!-- JDBC y driver según la BD real (por ejemplo, MySQL) -->
 <!-- Para las pruebas de integración usaremos H2 -->
 <dependency>
 <groupId>com.h2database</groupId>
 <artifactId>h2</artifactId>
 <version>2.2.224</version>
 <scope>test</scope>
 </dependency>

 <!-- JUnit 5 -->
 <dependency>
 <groupId>org.junit.jupiter</groupId>
 <artifactId>junit-jupiter</artifactId>
 <version>5.10.0</version>
 <scope>test</scope>
 </dependency>

 <!-- Mockito -->
 <dependency>
 <groupId>org.mockito</groupId>
 <artifactId>mockito-core</artifactId>
 <version>5.11.0</version>
 <scope>test</scope>
 </dependency>
</dependencies>
```

#### 3.1 Clase de dominio: `Venta`

```
// src/main/java/com/ejemplo/ventas/domain/Venta.java
package com.ejemplo.ventas.domain;

import java.time.LocalDate;

/**
 * Clase de dominio que representa una venta.
 *
 * Decisión de diseño:
 * - Mantener la clase como un POJO (Plain Old Java Object) con atributos,
 * constructor y getters/setters.
 * - No incluir lógica compleja aquí para separar responsabilidades:
 * la lógica de negocio principal se ubicará en el servicio.
 */
public class Venta {

 private Integer idVenta;
 private LocalDate fecha;
 private String cliente;
 private String producto;
 private int cantidad;
 private double precioUnitario;
 private double total;

 public Venta(Integer idVenta,
 LocalDate fecha,
 String cliente,
 String producto,
 int cantidad,
 double precioUnitario,
 double total) {
 this.idVenta = idVenta;
 this.fecha = fecha;
 this.cliente = cliente;
 this.producto = producto;
 this.cantidad = cantidad;
 this.precioUnitario = precioUnitario;
 this.total = total;
 }

 // Constructor sin id para nuevas ventas (id autogenerated en BD)
 public Venta(LocalDate fecha,
 String cliente,
 String producto,
 int cantidad,
 double precioUnitario) {
 this(null, fecha, cliente, producto, cantidad, precioUnitario, 0.0);
 }

 // Getters y setters
 public Integer getIdVenta() {
 return idVenta;
 }
}
```

```
public void setIdVenta(Integer idVenta) {
 this.idVenta = idVenta;
}

public LocalDate getFecha() {
 return fecha;
}

public void setFecha(LocalDate fecha) {
 this.fecha = fecha;
}

public String getCliente() {
 return cliente;
}

public void setCliente(String cliente) {
 this.cliente = cliente;
}

public String getProducto() {
 return producto;
}

public void setProducto(String producto) {
 this.producto = producto;
}

public int getCantidad() {
 return cantidad;
}

public void setCantidad(int cantidad) {
 this.cantidad = cantidad;
}

public double getPrecioUnitario() {
 return precioUnitario;
}

public void setPrecioUnitario(double precioUnitario) {
 this.precioUnitario = precioUnitario;
}

public double getTotal() {
 return total;
}

public void setTotal(double total) {
 this.total = total;
}
}
```

### 3.2 Interfaz de repositorio: `VentaRepository`

```
// src/main/java/com/ejemplo/ventas/repository/VentaRepository.java
package com.ejemplo.ventas.repository;

import com.ejemplo.ventas.domain.Venta;

import java.util.List;
import java.util.Optional;

/**
 * Interfaz de repositorio para la entidad Venta.
 *
 * Decisión de diseño:
 * - Definir una interfaz permite desacoplar la lógica de negocio (servicio)
 * de la implementación concreta de acceso a datos (JDBC, JPA, etc.).
 * - Facilita el uso de mocks en pruebas unitarias.
 */
public interface VentaRepository {

 Venta guardar(Venta venta);

 Optional<Venta> buscarPorId(Integer idVenta);

 List<Venta> listarTodas();

 Venta actualizar(Venta venta);

 void eliminar(Integer idVenta);
}
```

### 3.3 Implementación JDBC: `JdbcVentaRepository`

```
// src/main/java/com/ejemplo/ventas/repository/JdbcVentaRepository.java
package com.ejemplo.ventas.repository;

import com.ejemplo.ventas.domain.Venta;

import java.sql.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/**
 * Implementación de VentaRepository usando JDBC simple.
 *
 * Decisiones de diseño:
 * - Usar JDBC directo para que los estudiantes vean claramente la interacción
 * con la base de datos.
 * - Manejo de errores básico: se lanzan RuntimeException para simplificar

```

```
* el ejemplo, pero en un sistema real se usarían excepciones específicas
* y manejo más robusto.
*/
public class JdbcVentaRepository implements VentaRepository {

 private final String url;
 private final String user;
 private final String password;

 public JdbcVentaRepository(String url, String user, String password) {
 this.url = url;
 this.user = user;
 this.password = password;
 }

 private Connection getConnection() throws SQLException {
 return DriverManager.getConnection(url, user, password);
 }

 @Override
 public Venta guardar(Venta venta) {
 String sql = "INSERT INTO ventas (fecha, cliente, producto, cantidad,
precio_unitario, total) " +
 "VALUES (?, ?, ?, ?, ?, ?)";
 try (Connection conn = getConnection();
 PreparedStatement ps = conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {

 ps.setDate(1, Date.valueOf(venta.getFecha()));
 ps.setString(2, venta.getCliente());
 ps.setString(3, venta.getProducto());
 ps.setInt(4, venta.getCantidad());
 ps.setDouble(5, venta.getPrecioUnitario());
 ps.setDouble(6, venta.getTotal());

 ps.executeUpdate();

 try (ResultSet rs = ps.getGeneratedKeys()) {
 if (rs.next()) {
 venta.setIdVenta(rs.getInt(1));
 }
 }

 return venta;
 } catch (SQLException e) {
 throw new RuntimeException("Error al guardar venta", e);
 }
 }

 @Override
 public Optional<Venta> buscarPorId(Integer idVenta) {
 String sql = "SELECT id_venta, fecha, cliente, producto, cantidad,
precio_unitario, total " +
 "FROM ventas WHERE id_venta = ?";
```

```

 try (Connection conn = getConnection();
 PreparedStatement ps = conn.prepareStatement(sql)) {

 ps.setInt(1, idVenta);
 try (ResultSet rs = ps.executeQuery()) {
 if (rs.next()) {
 Venta venta = mapRowToVenta(rs);
 return Optional.of(venta);
 } else {
 return Optional.empty();
 }
 }
 } catch (SQLException e) {
 throw new RuntimeException("Error al buscar venta por id", e);
 }
 }

 @Override
 public List<Venta> listarTodas() {
 String sql = "SELECT id_venta, fecha, cliente, producto, cantidad,
precio_unitario, total FROM ventas";
 List<Venta> ventas = new ArrayList<>();
 try (Connection conn = getConnection();
 PreparedStatement ps = conn.prepareStatement(sql);
 ResultSet rs = ps.executeQuery()) {

 while (rs.next()) {
 ventas.add(mapRowToVenta(rs));
 }
 return ventas;
 } catch (SQLException e) {
 throw new RuntimeException("Error al listar ventas", e);
 }
 }

 @Override
 public Venta actualizar(Venta venta) {
 String sql = "UPDATE ventas SET fecha = ?, cliente = ?, producto = ?,
cantidad = ?, " +
 "precio_unitario = ?, total = ? WHERE id_venta = ?";
 try (Connection conn = getConnection();
 PreparedStatement ps = conn.prepareStatement(sql)) {

 ps.setDate(1, Date.valueOf(venta.getFecha()));
 ps.setString(2, venta.getCliente());
 ps.setString(3, venta.getProducto());
 ps.setInt(4, venta.getCantidad());
 ps.setDouble(5, venta.getPrecioUnitario());
 ps.setDouble(6, venta.getTotal());
 ps.setInt(7, venta.getIdVenta());

 ps.executeUpdate();
 return venta;
 } catch (SQLException e) {

```

```

 throw new RuntimeException("Error al actualizar venta", e);
 }
}

@Override
public void eliminar(Integer idVenta) {
 String sql = "DELETE FROM ventas WHERE id_venta = ?";
 try (Connection conn = getConnection();
 PreparedStatement ps = conn.prepareStatement(sql)) {

 ps.setInt(1, idVenta);
 ps.executeUpdate();
 } catch (SQLException e) {
 throw new RuntimeException("Error al eliminar venta", e);
 }
}

private Venta mapRowToVenta(ResultSet rs) throws SQLException {
 Integer id = rs.getInt("id_venta");
 LocalDate fecha = rs.getDate("fecha").toLocalDate();
 String cliente = rs.getString("cliente");
 String producto = rs.getString("producto");
 int cantidad = rs.getInt("cantidad");
 double precioUnitario = rs.getDouble("precio_unitario");
 double total = rs.getDouble("total");

 return new Venta(id, fecha, cliente, producto, cantidad, precioUnitario,
total);
}
}

```

### 3.4 Servicio de negocio: `VentaService`

```

// src/main/java/com/ejemplo/ventas/service/VentaService.java
package com.ejemplo.ventas.service;

import com.ejemplo.ventas.domain.Venta;
import com.ejemplo.ventas.repository.VentaRepository;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

/**
 * Servicio de negocio para gestionar ventas.
 *
 * Decisiones de diseño:
 * - Centralizar la lógica de negocio aquí: cálculo del total, validaciones
 * de cantidad y precio, reglas simples.
 * - El servicio depende de la interfaz VentaRepository, lo que permite
 * usar mocks en pruebas unitarias y distintas implementaciones en producción.
 */

```



```
public class VentaService {

 private final VentaRepository ventaRepository;

 public VentaService(VentaRepository ventaRepository) {
 this.ventaRepository = ventaRepository;
 }

 /**
 * Registra una nueva venta aplicando reglas de negocio básicas.
 * - Calcula el total = cantidad * precioUnitario.
 * - Valida que cantidad y precio sean positivos.
 */
 public Venta registrarVenta(LocalDate fecha,
 String cliente,
 String producto,
 int cantidad,
 double precioUnitario) {

 if (cantidad <= 0) {
 throw new IllegalArgumentException("La cantidad debe ser mayor que
cero");
 }
 if (precioUnitario <= 0) {
 throw new IllegalArgumentException("El precio unitario debe ser mayor
que cero");
 }

 double total = cantidad * precioUnitario;

 Venta venta = new Venta(fecha, cliente, producto, cantidad,
precioUnitario);
 venta.setTotal(total);

 return ventaRepository.guardar(venta);
 }

 /**
 * Obtiene una venta por su id.
 */
 public Optional<Venta> obtenerVentaPorId(Integer idVenta) {
 return ventaRepository.buscarPorId(idVenta);
 }

 /**
 * Lista todas las ventas.
 */
 public List<Venta> listarVentas() {
 return ventaRepository.listarTodas();
 }

 /**
 * Actualiza una venta existente.
 * Recalcula el total si cambian cantidad o precio.
 */
}
```

```

 */
 public Venta actualizarVenta(Venta venta) {
 if (venta.getCantidad() <= 0) {
 throw new IllegalArgumentException("La cantidad debe ser mayor que
cero");
 }
 if (venta.getPrecioUnitario() <= 0) {
 throw new IllegalArgumentException("El precio unitario debe ser mayor
que cero");
 }

 double total = venta.getCantidad() * venta.getPrecioUnitario();
 venta.setTotal(total);

 return ventaRepository.actualizar(venta);
 }

 /**
 * Elimina una venta por id.
 */
 public void eliminarVenta(Integer idVenta) {
 ventaRepository.eliminar(idVenta);
 }
}

```

### 3.5 Clase Main de ejemplo de uso

```

// src/main/java/com/ejemplo/ventas/Main.java
package com.ejemplo.ventas;

import com.ejemplo.ventas.domain.Venta;
import com.ejemplo.ventas.repository.JdbcVentaRepository;
import com.ejemplo.ventas.service.VentaService;

import java.time.LocalDate;
import java.util.List;

/**
 * Clase Main que muestra un flujo simple de uso del servicio.
 *
 * Decisión didáctica:
 * - No se implementa una interfaz gráfica ni API REST; el objetivo es
 * tener un punto de entrada que los estudiantes puedan ejecutar
 * para ver el comportamiento básico y relacionarlo con las pruebas.
 */
public class Main {

 public static void main(String[] args) {
 // Configuración simple de conexión (ejemplo; ajustar según la BD real)
 String url = "jdbc:mysql://localhost:3306/bd_ventas";
 String user = "root";
 String password = "root";
 }
}

```

```
JdbcVentaRepository repository = new JdbcVentaRepository(url, user,
password);
VentaService service = new VentaService(repository);

// Registrar una venta de ejemplo
Venta nuevaVenta = service.registrarVenta(
 LocalDate.now(),
 "Cliente Demo",
 "Producto Demo",
 5,
 10.0
);

System.out.println("Venta registrada con ID: " + nuevaVenta.getIdVenta()
 + " y total: " + nuevaVenta.getTotal());

// Listar todas las ventas
List<Venta> ventas = service.listarVentas();
System.out.println("Ventas registradas:");
for (Venta v : ventas) {
 System.out.println("ID: " + v.getIdVenta() + " - Cliente: " +
v.getCliente()
 + " - Total: " + v.getTotal());
}
}
```

## 4. Pruebas unitarias con JUnit 5 y Mockito

### 4.1 Propósito de las pruebas unitarias

- **Propósito:** Validar la lógica de negocio de manera aislada, sin depender de la base de datos ni de otros componentes externos.
- **En este proyecto:** Se prueban los métodos de `VentaService` usando un mock de `VentaRepository`.
- **Herramientas usadas:** JUnit 5, Mockito.
- **Cómo ejecutarlas:**
  - Con Maven: `mvn test` (si todas las pruebas están en el mismo módulo).
  - Se pueden agrupar por paquetes (`unit`, `integration`, `acceptance`) para fines didácticos.

### 4.2 Clase de pruebas unitarias: `VentaServiceTest`

```
// src/test/java/com/ejemplo/ventas/service/VentaServiceTest.java
package com.ejemplo.ventas.service;

import com.ejemplo.ventas.domain.Venta;
import com.ejemplo.ventas.repository.VentaRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.ArgumentCaptor;
```

```
import java.time.LocalDate;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

/**
 * Pruebas unitarias de VentaService.
 *
 * Decisión didáctica:
 * - Usar Mockito para simular el comportamiento del repositorio.
 * - No se toca la base de datos real; el foco está en la lógica de negocio.
 */
public class VentaServiceTest {

 private VentaRepository ventaRepository;
 private VentaService ventaService;

 @BeforeEach
 void setUp() {
 // Creamos un mock del repositorio
 ventaRepository = mock(VentaRepository.class);
 ventaService = new VentaService(ventaRepository);
 }

 @Test
 void registrarVenta_calculaTotalCorrectamente() {
 // Given: datos de una venta válida
 LocalDate fecha = LocalDate.of(2024, 1, 1);
 String cliente = "Cliente A";
 String producto = "Producto X";
 int cantidad = 3;
 double precioUnitario = 10.0;

 // Configuramos el mock para devolver la venta que se le pasa
 // simulando que la guarda correctamente.
 ArgumentCaptor<Venta> captor = ArgumentCaptor.forClass(Venta.class);
 when(ventaRepository.guardar(any(Venta.class)))
 .thenAnswer(invocation -> invocation.getArgument(0));

 // When: se registra la venta
 Venta ventaRegistrada = ventaService.registrarVenta(
 fecha, cliente, producto, cantidad, precioUnitario);

 // Then: el total debe ser cantidad * precioUnitario
 assertEquals(30.0, ventaRegistrada.getTotal(), 0.001);

 // Verificamos que el repositorio fue llamado con una venta cuyo total es
 // correcto
 verify(ventaRepository).guardar(captor.capture());
 Venta ventaGuardada = captor.getValue();
 assertEquals(30.0, ventaGuardada.getTotal(), 0.001);
 }
}
```

```
// Comentario didáctico:
// Esta es una prueba unitaria porque solo ejercita la lógica de
VentaService
// y el repositorio está simulado (mock), sin acceso real a la BD.
}

@Test
void registrarVenta_conCantidadNoValida_lanzaExcepcion() {
 // Given: cantidad inválida (0)
 LocalDate fecha = LocalDate.now();
 String cliente = "Cliente B";
 String producto = "Producto Y";
 int cantidad = 0;
 double precioUnitario = 10.0;

 // When & Then: se espera una IllegalArgumentException
 assertThrows(IllegalArgumentException.class, () ->
 ventaService.registrarVenta(fecha, cliente, producto, cantidad,
precioUnitario));

 // Verificamos que el repositorio nunca fue llamado
 verify(ventaRepository, never()).guardar(any(Venta.class));

 // Comentario didáctico:
 // Esta prueba valida una regla de negocio (cantidad > 0) de forma aislada.
}

@Test
void obtenerVentaPorId_devuelveVentaCuandoExiste() {
 // Given: una venta existente en el repositorio (simulada)
 Venta venta = new Venta(1, LocalDate.now(), "Cliente C",
 "Producto Z", 2, 15.0, 30.0);
 when(ventaRepository.buscarPorId(1)).thenReturn(Optional.of(venta));

 // When: se solicita la venta por id
 Optional<Venta> resultado = ventaService.obtenerVentaPorId(1);

 // Then: se obtiene la venta esperada
 assertTrue(resultado.isPresent());
 assertEquals("Cliente C", resultado.get().getCliente());

 // Comentario didáctico:
 // Aunque esta prueba toca el repositorio, sigue siendo unitaria porque
 // el repositorio es un mock y no hay interacción real con la BD.
}
}
```

---

## 5. Pruebas de integración con JUnit 5 y H2

### 5.1 Propósito de las pruebas de integración

- **Propósito:** Validar la interacción entre componentes, por ejemplo, servicio + repositorio + base de datos.

- **En este proyecto:** Se ejercita `VentaService` junto con `JdbcVentaRepository` contra una base de datos H2 en memoria.
- **Herramientas usadas:** JUnit 5, H2 (base de datos en memoria), JDBC.
- **Cómo ejecutarlas:**
  - Con Maven: `mvn test`.
  - Opcionalmente, se pueden ubicar en un paquete `integration` para diferenciarlas.

## 5.2 Clase de pruebas de integración: `VentaServiceIntegrationTest`

```
// src/test/java/com/ejemplo/ventas/integration/VentaServiceIntegrationTest.java
package com.ejemplo.ventas.integration;

import com.ejemplo.ventas.domain.Venta;
import com.ejemplo.ventas.repository.JdbcVentaRepository;
import com.ejemplo.ventas.service.VentaService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.time.LocalDate;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

/**
 * Pruebas de integración para VentaService + JdbcVentaRepository + H2.
 *
 * Decisión didáctica:
 * - Usar H2 en memoria para tener una BD real pero efímera, ideal para pruebas.
 * - Ejecutar el script de creación de tabla dentro del setUp.
 */
public class VentaServiceIntegrationTest {

 private VentaService ventaService;

 @BeforeEach
 void setUp() throws Exception {
 // Configuración de H2 en memoria
 String url = "jdbc:h2:mem:bd_ventas_test;DB_CLOSE_DELAY=-1";
 String user = "sa";
 String password = "";

 // Crear tabla 'ventas' en H2 (similar a bd_ventas.sql)
 try (Connection conn = DriverManager.getConnection(url, user, password);
 Statement stmt = conn.createStatement()) {

 stmt.execute("""
 CREATE TABLE ventas (
 id_venta INT AUTO_INCREMENT PRIMARY KEY,
 fecha DATE NOT NULL,

```

```
 cliente VARCHAR(100) NOT NULL,
 producto VARCHAR(100) NOT NULL,
 cantidad INT NOT NULL,
 precio_unitario DOUBLE NOT NULL,
 total DOUBLE NOT NULL
)
 """);
}

JdbcVentaRepository repository = new JdbcVentaRepository(url, user,
password);
ventaService = new VentaService(repository);
}

@Test
void registrarYListarVentas_integraServicioYRepositorio() {
 // Given: una venta válida
 Venta v1 = ventaService.registrarVenta(
 LocalDate.of(2024, 1, 1),
 "Cliente Integración",
 "Producto I",
 2,
 20.0
);

 // When: se listan las ventas
 List<Venta> ventas = ventaService.listarVentas();

 // Then: la venta registrada debe estar en la lista
 assertFalse(ventas.isEmpty());
 assertEquals(1, ventas.size());
 assertEquals(v1.getIdVenta(), ventas.get(0).getIdVenta());
 assertEquals(40.0, ventas.get(0).getTotal(), 0.001);

 // Comentario didáctico:
 // Esta es una prueba de integración porque:
 // - Usa la implementación real JdbcVentaRepository.
 // - Interactúa con una BD H2 real (aunque en memoria).
 // - Verifica el flujo completo servicio + repositorio + BD.
}

@Test
void actualizarVenta_persisteCambiosEnLaBD() {
 // Given: una venta registrada
 Venta v1 = ventaService.registrarVenta(
 LocalDate.of(2024, 2, 1),
 "Cliente Integración 2",
 "Producto J",
 1,
 50.0
);

 // Modificamos cantidad y precio
 v1.setCantidad(3);
```

```
 v1.setPrecioUnitario(30.0);

 // When: se actualiza la venta
 Venta actualizada = ventaService.actualizarVenta(v1);

 // Then: el total debe reflejar los nuevos valores
 assertEquals(90.0, actualizada.getTotal(), 0.001);

 // Y al leer desde la BD, los cambios deben persistir
 List<Venta> ventas = ventaService.listarVentas();
 assertEquals(1, ventas.size());
 assertEquals(90.0, ventas.get(0).getTotal(), 0.001);

 // Comentario didáctico:
 // Esta prueba verifica que la actualización se refleja en la BD,
 // integrando servicio, repositorio y H2.
 }
}
```

## 6. Pruebas de aceptación (BDD: Given–When–Then)

### 6.1 Propósito de las pruebas de aceptación

- **Propósito:** Validar que el sistema cumple criterios de usuario o de negocio, expresados en lenguaje cercano al usuario.
- **En este proyecto:** Se definen casos en estilo BDD (Given–When–Then) usando JUnit 5, sin necesidad de Cucumber.
- **Herramientas usadas:** JUnit 5, H2 (para tener un entorno real), `VentaService` + `JdbcVentaRepository`.
- **Cómo ejecutarlas:**
  - Con Maven: `mvn test`.
  - Se pueden ubicar en un paquete `acceptance` para diferenciarlas.

### 6.2 Criterios de usuario de ejemplo

#### 1. Criterio 1:

*Como usuario de negocio quiero registrar una venta válida y ver el total calculado correctamente para asegurar que los reportes financieros sean confiables.*

#### 2. Criterio 2:

*Como usuario de negocio quiero que el sistema rechace ventas con cantidad o precio no válidos para evitar errores en la facturación.*

### 6.3 Clase de pruebas de aceptación: `VentaAcceptanceTest`

```
// src/test/java/com/ejemplo/ventas/acceptance/VentaAcceptanceTest.java
package com.ejemplo.ventas.acceptance;

import com.ejemplo.ventas.domain.Venta;
```



```
import com.ejemplo.ventas.repository.JdbcVentaRepository;
import com.ejemplo.ventas.service.VentaService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.time.LocalDate;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

/**
 * Pruebas de aceptación en estilo BDD (Given-When-Then).
 *
 * Decisión didáctica:
 * - Usar JUnit 5 con comentarios y nombres de métodos que reflejen
 * la estructura Given-When-Then.
 * - No se usa Cucumber para mantener el ejemplo simple.
 */
public class VentaAcceptanceTest {

 private VentaService ventaService;

 @BeforeEach
 void setUp() throws Exception {
 // Configuración de H2 en memoria para simular el entorno de producción
 String url = "jdbc:h2:mem:bd_ventas_acceptance;DB_CLOSE_DELAY=-1";
 String user = "sa";
 String password = "";

 try (Connection conn = DriverManager.getConnection(url, user, password);
 Statement stmt = conn.createStatement()) {

 stmt.execute("""
 CREATE TABLE ventas (
 id_venta INT AUTO_INCREMENT PRIMARY KEY,
 fecha DATE NOT NULL,
 cliente VARCHAR(100) NOT NULL,
 producto VARCHAR(100) NOT NULL,
 cantidad INT NOT NULL,
 precio_unitario DOUBLE NOT NULL,
 total DOUBLE NOT NULL
)
 """);
 }

 JdbcVentaRepository repository = new JdbcVentaRepository(url, user,
password);
 ventaService = new VentaService(repository);
 }

 @Test
```

```
void givenVentaValida_whenSeRegistra_thenTotalEsCorrectoYLaVentaEsVisible() {
 // Given: una venta válida según el criterio de usuario
 LocalDate fecha = LocalDate.of(2024, 3, 1);
 String cliente = "Cliente BDD";
 String producto = "Producto BDD";
 int cantidad = 4;
 double precioUnitario = 25.0;

 // When: el usuario registra la venta en el sistema
 Venta ventaRegistrada = ventaService.registrarVenta(
 fecha, cliente, producto, cantidad, precioUnitario);

 // Then: el total calculado debe ser correcto y la venta debe aparecer en
 los listados
 assertEquals(100.0, ventaRegistrada.getTotal(), 0.001);

 List<Venta> ventas = ventaService.listarVentas();
 assertFalse(ventas.isEmpty());
 assertEquals(1, ventas.size());
 assertEquals(100.0, ventas.get(0).getTotal(), 0.001);

 // Comentario BDD:
 // Given: existe una venta válida con cantidad y precio positivos.
 // When: el usuario registra la venta.
 // Then: el sistema calcula correctamente el total y la venta se refleja en
 los reportes.
}

@Test
void
givenVentaConCantidadInvalida_whenSeIntentaRegistrar_thenElSistemaLaRechaza() {
 // Given: una venta con cantidad inválida (0), que viola el criterio de
 negocio
 LocalDate fecha = LocalDate.of(2024, 3, 2);
 String cliente = "Cliente BDD 2";
 String producto = "Producto BDD 2";
 int cantidad = 0;
 double precioUnitario = 30.0;

 // When: el usuario intenta registrar la venta
 // Then: el sistema debe rechazarla lanzando una excepción y no debe
 aparecer en los listados
 assertThrows(IllegalArgumentException.class, () ->
 ventaService.registrarVenta(fecha, cliente, producto, cantidad,
 precioUnitario));

 List<Venta> ventas = ventaService.listarVentas();
 assertTrue(ventas.isEmpty());

 // Comentario BDD:
 // Given: una venta con datos inválidos.
 // When: el usuario intenta registrarla.
 // Then: el sistema la rechaza para proteger la calidad de la información.
```

```
}
}
```

## 7. Resumen por tipo de prueba

### 7.1 Pruebas unitarias

- **Propósito:** Validar la lógica de negocio de forma aislada, sin dependencias externas reales.
- **Herramientas usadas:** JUnit 5, Mockito.
- **Cómo ejecutarlas:**
  - Ejecutar `mvn test` (todas las pruebas).
  - Opcionalmente, usar convenciones de nombres o paquetes (`service`, `unit`) para identificarlas.
- **Feedback al equipo:**
  - Rápido y granular.
  - Indica qué métodos o reglas de negocio fallan.
  - Ideal para TDD y refactorización segura.

### 7.2 Pruebas de integración

- **Propósito:** Verificar que los componentes colaboran correctamente (servicio + repositorio + BD).
- **Herramientas usadas:** JUnit 5, H2, JDBC.
- **Cómo ejecutarlas:**
  - `mvn test`.
  - Se pueden ubicar en `src/test/java/com/ejemplo/ventas/integration`.
- **Feedback al equipo:**
  - Muestra problemas de configuración, mapeo de datos, SQL, etc.
  - Más lentas que las unitarias, pero más cercanas al entorno real.

### 7.3 Pruebas de aceptación (BDD)

- **Propósito:** Validar criterios de usuario/negocio expresados en lenguaje Given–When–Then.
- **Herramientas usadas:** JUnit 5, H2, `VentaService` + `JdbcVentaRepository`.
- **Cómo ejecutarlas:**
  - `mvn test`.
  - Ubicadas en `src/test/java/com/ejemplo/ventas/acceptance`.
- **Feedback al equipo:**
  - Indica si el sistema cumple las expectativas del usuario.
  - Útiles para comunicación entre negocio y equipo técnico.

## 8. Tabla comparativa de tipos de prueba

Tipo de prueba	Objetivo principal	Alcance	Dependencias	Velocidad de ejecución	Ejemplos en este proyecto
Unitaria	Validar lógica de negocio	Métodos individuales	Mocks de repositorios,	Muy alta	<code>VentaServiceTest</code> (cálculo de total, validaciones)

Tipo de prueba	Objetivo principal	Alcance	Dependencias	Velocidad de ejecución	Ejemplos en este proyecto
	de forma aislada	o clases	sin BD real		
Integración	Verificar interacción entre componentes	Servicio + repositorio + BD	BD H2 en memoria, JDBC real	Media	<code>VentaServiceIntegrationTest</code> (registro y actualización)
Aceptación (BDD)	Validar criterios de usuario en lenguaje Given–When–Then	Flujo funcional completo	Servicio + repositorio + BD H2	Menor que unitarias	<code>VentaAcceptanceTest</code> (registro válido, rechazo de venta inválida)